

Simulação de Incêndio Florestal: Implementação Sequencial, Paralela e Distribuída

Disciplina: Sistemas Distribuídos

Alunos: Ricardo Lima, Vinícius Farias, Davi Zeneratto, Geisily Vargas

Professor: Rogério Santos Pozza

1. Introdução

A propagação de incêndios florestais é um fenômeno dinâmico, complexo e altamente dependente das condições locais. Modelos computacionais permitem simular esse comportamento e compreender como um fogo se espalha em diferentes situações.

Neste trabalho, implementamos uma simulação de incêndio florestal em três abordagens distintas:

- Versão Sequencial
- Versão Paralela com threads
- Versão Distribuída com sockets

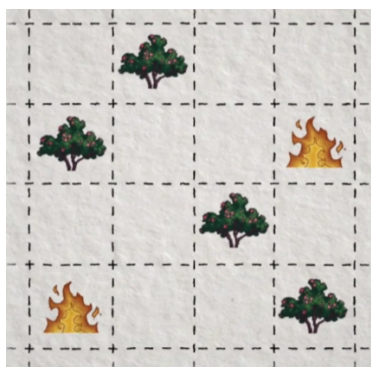
O objetivo principal é comparar desempenho, escalabilidade e limitações de cada implementação, observando como o mesmo problema se comporta em arquiteturas de execução diferentes.

2. Explicação do problema (Forest Fire)

O modelo utilizado representa uma floresta como uma matriz 2D, na qual cada célula pode estar em um dos três estados:

- 0 - vazio
- 1 - árvore
- 2 - queimando

Exemplo:



3. Descrição da Proposta de Solução

3.1 Versão Sequencial

A versão sequencial serve como base para comparação de desempenho. Aqui, toda a matriz é processada em um único fluxo de execução (um único thread). A simplicidade dessa versão garante resultados consistentes e previsíveis.

Esta solução foi baseada em fontes públicas encontradas na internet e aprimorada para os testes necessários para o trabalho com auxílio de ferramentas IA generativa.

3.2 Versão Paralela (Threads)

A versão paralela Divide a matriz igualmente entre várias threads.

- Cada thread simula sua faixa da grade.
- Não há comunicação entre threads durante a iteração (grade é atualizada de forma sincronizada).
- Objetivo: reduzir o tempo de execução em hardware multicore.

3.3 Versão Distribuída (Sockets)

A implementação distribuída utiliza comunicação em rede via sockets TCP, organizada em uma arquitetura cliente-servidor, em que o servidor divide a matriz e coordena a simulação, e cada cliente executa a atualização da sua submatriz local, enviando os resultados de volta ao servidor.

Arquitetura Geral:

-Servidor:

- Aceita conexões de múltiplos clientes.
- Divide a matriz global em faixas horizontais.
- Envia a cada cliente sua respectiva submatriz inicial.
- Realiza a sincronização implícita entre clientes (por troca de mensagens).
- Mede o tempo total da simulação distribuída.

-Cliente:

- Recebem uma submatriz e parâmetros da simulação.
- A cada iteração:
 - recebem do servidor as bordas (linha superior e inferior),
 - atualizam sua submatriz local,
 - enviam o resultado atualizado de volta ao servidor.
- Ao final, encerra a conexão.

4. Configuração dos Experimentos

Hardware utilizado (Distribuído)

- CPU: Intel Core i5-11400H
- Núcleos físicos e lógicos: 6
- RAM: 8GB
- Sistema Operacional: Windows

Configuração do Colab (Paralelo e Sequencial)

- CPU Intel Xeon virtualizada
- 2 vCPUs
- 12 GB RAM

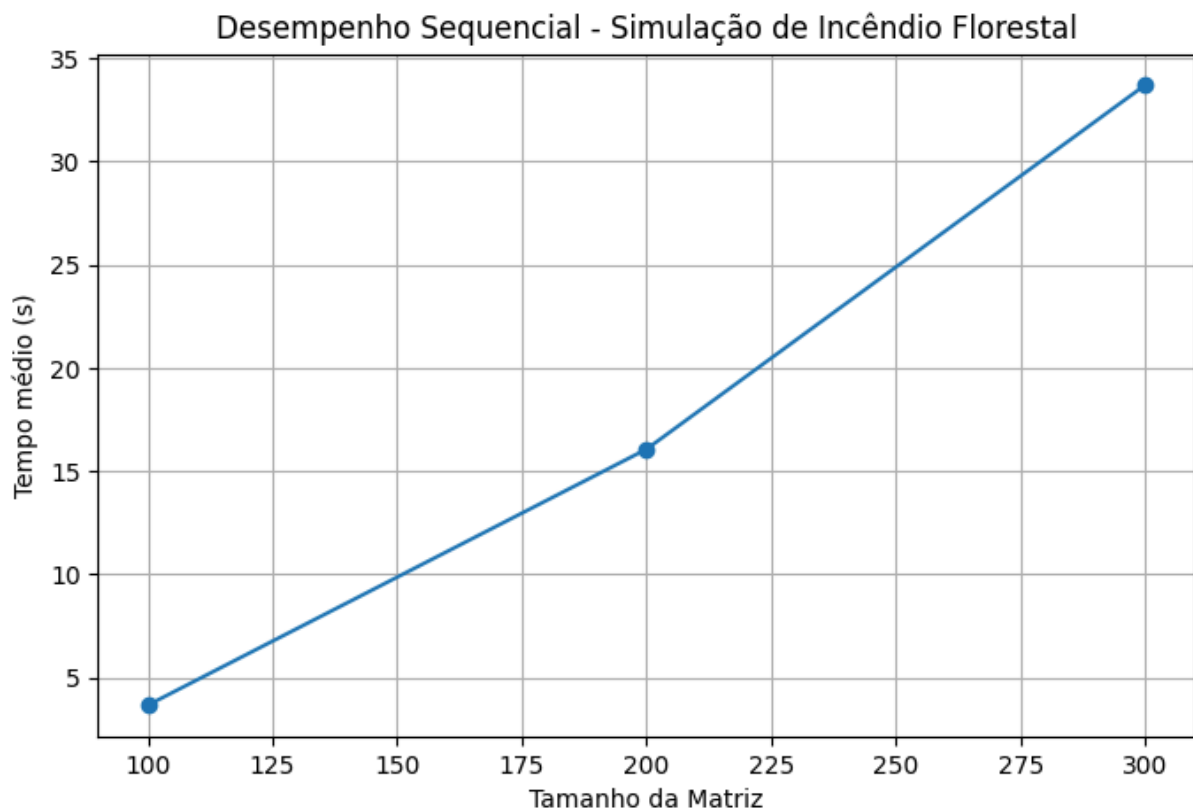
5. Resultados Experimentais

5.1 Tabela comparativa geral (Sequencial x Paralelo x Distribuído)

Versão	Tamanho	Iterações	Threads/Hosts	Tempo médio
Sequencial	100x100	100	1	3.6613
Sequencial	200x200	100	1	16.0653
Sequencial	300x300	100	1	33.7108
Paralela	100x100	100	1 – 4 threads	4,0336
Paralela	200x200	100	1 – 4 threads	18,0066
Paralela	300x300	100	1 – 4 threads	30,8955
Distribuído	100x100	300	2 – 4 nós	2,4578
Distribuído	200x200	300	2 – 4 nós	5,0724
Distribuído	300x300	300	2 – 4 nós	7,9522

5.2 Gráficos

Gráfico 1 - Desempenho Sequencial



Resultados finais sequencial:

Execução	Tamanho	Tempo(s)
1	100x100	3.630641
2	100x100	3.534923
3	100x100	3.818472
4	200x200	15.955237
5	200x200	17.259954
6	200x200	14.980610
7	300x300	35.853840
8	300x300	30.772537
9	300x300	34.506144

Gráfico 2 - Desempenho Paralelo

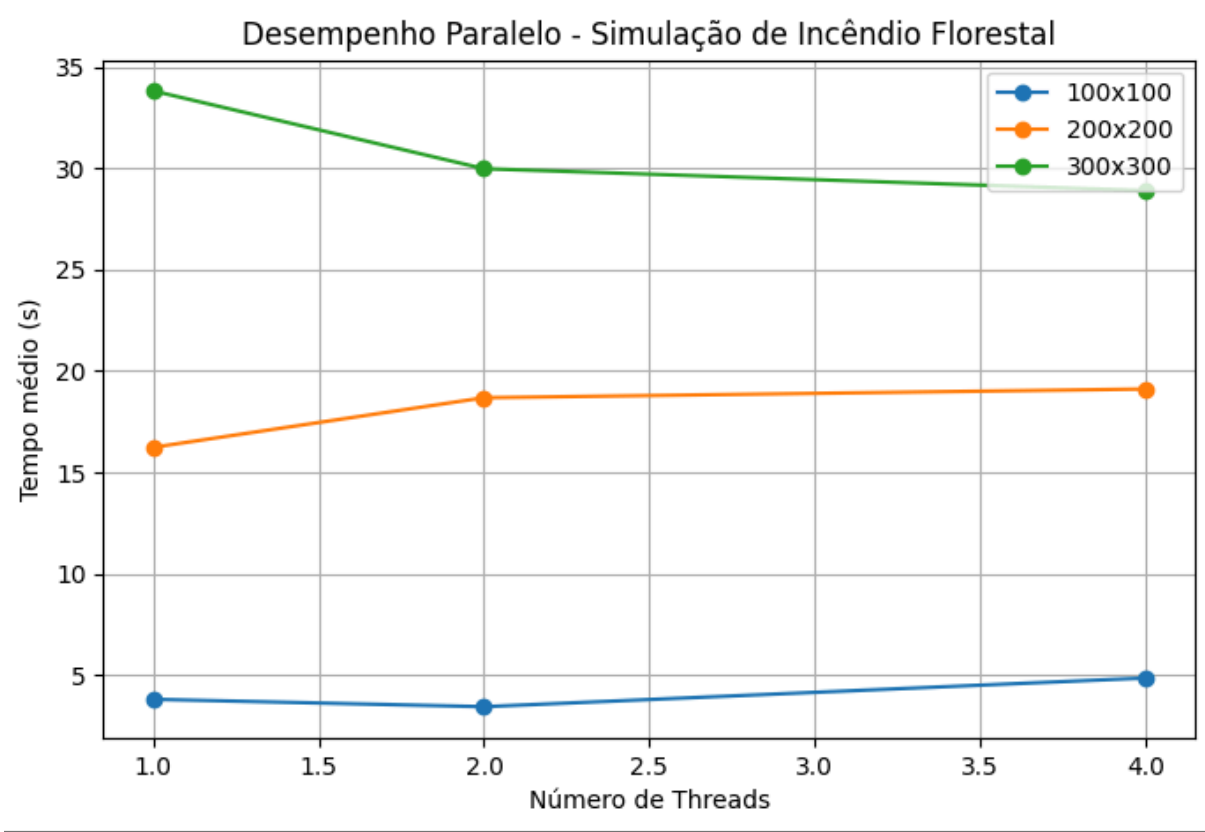


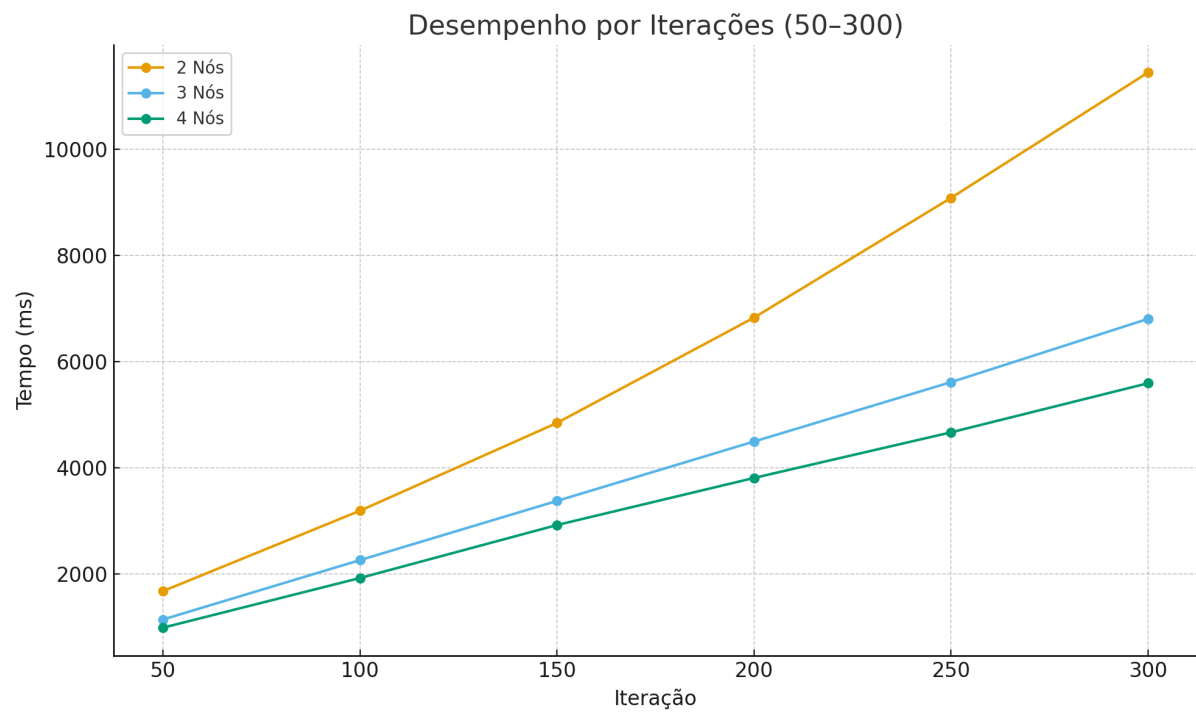
Tabela 1 - Speedup Paralelo

Tamanho	Threads	Tempo(s)	Speedup	Eficiência(%)
100x100	1	3.804623	0.9623	96.23
100x100	2	3.440416	1.0639	53.20
100x100	4	4.856059	0.7539	18.85
200x200	1	16.230860	0.9900	99.00
200x200	2	18.680015	0.8599	42.99
200x200	4	19.109277	0.8413	21.03
300x300	1	33.815709	0.9968	99.68
300x300	2	29.975246	1.1243	56.21
300x300	4	28.895726	1.1666	29.17

Resultados finais paralelo:

Execução	Tamanho	Threads	Tempo(s)
1	100x100	1	3.804623
2	100x100	2	3.440416
3	100x100	4	4.856059
4	200x200	1	16.230860
5	200x200	2	18.680015
6	200x200	4	19.109277
7	300x300	1	33.815709
8	300x300	2	29.975246
9	300x300	4	28.895726

Gráfico 3 - Desempenho Distribuído



Resultados finais distribuído:

Execução	Tamanho	Nós	Tempo
1	100x100	2	870.97 ms
2	100x100	3	681.17 ms
3	100x100	4	561.50 ms
4	200x200	2	2943.10 ms
5	200x200	3	2207.20 ms
6	200x200	4	1974.10 ms
7	300x300	2	11453.46ms
8	300x300	3	6808.36ms
9	300x300	4	5594.78ms

6. Comparação entre as Propostas

6.1 Versão Sequencial

A versão sequencial serviu como ponto de partida para avaliar o desempenho das outras duas versões.

Ela apresenta um comportamento esperado: conforme o tamanho da matriz aumenta, o tempo de execução cresce bastante.

Apesar disso, ela é importante porque fornece uma base sólida para calcular speedup e eficiência.

6.2 Versão Paralela (Threads)

A versão paralela apresentou melhorias em relação ao sequencial, especialmente quando usamos 2 threads, onde o ganho foi mais evidente.

Porém, ao aumentar para 4 threads, o desempenho não melhorou como esperado. Isso acontece por alguns motivos:

- As threads precisam acessar a mesma matriz, o que gera pequenas disputas na memória.
- A cada iteração todas as threads precisam sincronizar, o que adiciona atraso.
- O hardware limita o quanto o paralelismo realmente pode ajudar.

Mesmo com essas limitações, a versão paralela ainda é útil para acelerar o processamento em matrizes pequenas e médias.

6.3 Versão Distribuída (Sockets)

A versão distribuída apresentou os melhores resultados dentre todas as abordagens, e isso se explica diretamente pela forma como o código cliente-servidor foi implementado. Diferentemente da versão paralela baseada em threads, onde há compartilhamento de memória, na solução distribuída cada cliente é um processo independente, executando sua parte da simulação de maneira isolada e sem disputar recursos internos.

A comunicação entre servidor e clientes é simples, estruturada e eficiente: a cada iteração, o servidor envia apenas duas linhas de borda (superior e inferior) para cada cliente, e o cliente retorna apenas sua submatriz atualizada. Esse fluxo de troca de mensagens é mínimo e previsível, tornando o overhead de comunicação muito baixo.

7. Escalabilidade e Eficiência

7.1 Sequencial

Não escala, pois utiliza apenas um núcleo do processador.
Foi usado apenas como referência.

7.2 Paralelo

Escala parcialmente.

Funciona bem com 2 threads, mas perde eficiência com 4 threads porque:

- Várias threads acessam ao mesmo tempo a mesma matriz, gerando esperas;
- Existe sincronização obrigatória a cada iteração;
- O processador precisa revezar entre as threads.

Por isso, o speedup não cresce na mesma proporção do número de threads.

7.3 Distribuído

A versão distribuída foi a abordagem que demonstrou a melhor escalabilidade e isso ocorre devido à estrutura do sistema implementado. Diferentemente das threads, cada cliente é um processo independente, que executa sua parte do cálculo sem disputar memória ou competir por recursos internos do interpretador Python.

8. Contribuição Individual dos Membros do Grupo

Geisily: Contribuiu com a organização dos dados experimentais, montagem das tabelas e análise comparativa entre as três abordagens. Auxiliou na estruturação metodológica e na interpretação dos gráficos.

Ricardo: Atuou no desenvolvimento da versão paralela utilizando threads em Python. Implementou a divisão da matriz entre as threads, ajustou o fluxo de sincronização e realizou testes para verificar o desempenho. Auxiliou na análise dos speedups e na interpretação dos resultados obtidos.

Vinícius: Responsável pela criação da versão distribuída cliente-servidor usando sockets. Implementou tanto o servidor quanto os clientes, a lógica de envio de bordas, reconstrução da matriz global e coordenação das iterações. Também conduziu testes de escalabilidade e analisou o comportamento distribuído em relação às demais abordagens.

Davi: Foi responsável pela implementação inicial da versão sequencial do modelo de simulação, pesquisando em fontes públicas e garantindo a funcionalidade básica do algoritmo e validando a lógica de propagação do incêndio. Elaborou boa parte do texto explicativo do relatório e revisou o conteúdo final.

A combinação das diferentes contribuições permitiu que o trabalho fosse desenvolvido de forma eficiente e completa, integrando implementação, experimentação e análise crítica. Todos os membros participaram de discussões, decisões técnicas e validações ao longo do processo.

9. Desafios e Soluções

Durante o desenvolvimento das três abordagens da simulação, diversos desafios técnicos surgiram, tanto no nível de implementação quanto no de desempenho.

9.1 Necessidade de comunicação eficiente no sistema distribuído

A versão distribuída exigiu a criação de um protocolo de comunicação robusto entre servidor e clientes. O envio de objetos Python por socket envolve fragmentação de dados, risco de leitura parcial e necessidade de reconstrução exata das fronteiras da matriz.

Solução:

Foi implementado um protocolo próprio de envio e recebimento baseado em:

- Serialização com **pickle**,
- Cabeçalho contendo o tamanho do payload (8 bytes),
- Funções seguras de leitura (**recv_all**) garantindo recebimento integral,
- Uso de mensagens estruturadas contendo **top**, **bottom** e **submatriz**.

O protocolo eliminou erros de sincronização e garantiu estabilidade mesmo com múltiplos clientes.

9.2 Controle de aleatoriedade

O uso direto de **random.random()** e **np.random.rand()** resultava em comportamento não reprodutível e pouco consistente entre execuções, dificultando análises comparativas.

Solução:

Foram adotadas práticas como:

- Sementes fixas para inicializacao do grid,
- Geradores distintos para cada thread,
- Semente variável nos clientes distribuídos para evitar sincronização artificial.

9.3 Sincronização e consistência na versão paralela

No início, as threads compartilhavam tanto a matriz original quanto a matriz de saída, o que causava interferência entre atualizações simultâneas.

Solução:

- Usar a matriz original **grid** apenas para leitura,
- Usar uma matriz separada **nov** para escrita,
- Sincronizar todas as threads com **join()** ao final de cada iteração,
- Criar um gerador aleatório independente por thread para evitar correlação de eventos aleatórios.

10. Conclusão

O desenvolvimento das três versões do simulador: sequencial, paralela com threads e distribuída via sockets, permitiu uma compreensão aprofundada sobre o impacto de diferentes modelos de paralelismo no desempenho de aplicações computacionais.

- A versão sequencial serviu como referência.
- Na versão paralela trouxe ganhos razoáveis para matrizes menores, mas esbarrou em limitações de memória e sincronização.
- Já a versão distribuída apresentou os melhores resultados, especialmente em matrizes maiores, mostrando-se a opção mais eficiente para problemas de maior escala.

Em síntese, os resultados obtidos mostram que cada abordagem apresenta vantagens específicas de acordo com o cenário de uso. A versão sequencial é adequada para testes simples, validação do modelo e execução em máquinas com recursos limitados, servindo como referência básica de desempenho. Já a versão paralela com threads se mostra útil apenas em situações onde há poucos núcleos disponíveis e o tamanho da matriz é relativamente pequeno. Por fim, a versão distribuída se destaca como a mais eficiente para grandes volumes de dados e ambientes com múltiplos processos ou máquinas, uma vez que permite paralelismo real, minimiza a competição por memória e escala de forma mais consistente.

Assim, o estudo evidencia que não existe uma solução única que seja superior em todos os casos. cada técnica é mais adequada a um contexto específico, dependendo do tamanho do problema e da arquitetura disponível.

11. Link GitHub

<https://github.com/lodestaarz/PROJETO-SistemasDistribuidos>

12. Referências

The Big Book of Small Python Projects - Forest Fire Sim. Disponível em:

<https://inventwithpython.com/bigbookpython/project29.html>

Python project: Forest fire simulation tutorial with PyGame. YouTube. Disponível em:

<https://www.youtube.com/watch?v=o5ALfgFhYkQ>

Ferramentas de IA generativa, como ChatGPT, foram utilizadas.